나# 정적 분석 + 보안 스캔 도구 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

코드를 실행하지 않고 읽기만 해서 버그/보안 취약점을 찾는 도구들.

---

1. 비유

동적 분석 (실행해서 테스트): 음식을 실제로 먹어봐서 맛을 확인 정적 분석 (코드를 읽어서 검사): 레시피만 보고 "이 재료 조합은 위험하다"고 판단

정적 분석은 코드를 실행하지 않으므로 빠르고, PR 단계에서 바로 검사 가능.

---

2. Dev 2에서 사용하는 도구 5가지

Pylint — 코드 품질 검사

```
pylint src/auth.py --output-format=json
```

- 무엇을: 문법 오류, 미사용 변수, 네이밍 규칙 위반
- 결과: 각 이슈에 `message`, `line`, `symbol(rule_id)` 반환
- **Dev 2 severity**: 대부분 Minor (코드 스멜)

```
[{"type": "warning", "line": 10, "message": "Unused variable 'x'", "symbol":
"unused-variable"}]
```

---

radon — 순환복잡도 측정

```
radon cc src/auth.py -j
```

- 무엇을: 함수의 복잡도(분기 수) 측정. 숫자가 클수록 복잡
- 기준: 복잡도 > 10 → Major, > 20 → Critical
- **Dev 2** 의미: "이 함수는 테스트하기 어렵고 버그가 숨기 쉽다"

```
복잡도 5  → A (간단)
복잡도 15 → C (복잡, 리팩토링 권고)
복잡도 25 → F (매우 복잡, 즉시 분리)
```

Bandit — **Python** 보안 취약점 스캔

```
bandit -r src/ -f json
```

- 무엇을: SQL Injection, 하드코딩된 비밀번호, 안전하지 않은 함수 사용
- 결과: `issue_severity`(HIGH/MEDIUM/LOW) + `issue_confidence`

- **Dev 2** 매핑: HIGH ⟶ Critical, MEDIUM ⟶ Major, LOW ⟶ Minor


```
{
  "issue_severity": "HIGH",
  "issue_text": "Possible SQL injection via string-based query construction",
  "line_number": 42,
  "filename": "src/auth.py"
}
```

Semgrep — OWASP 룰셋 기반 패턴 검사

```
semgrep --config=p/owasp-top-ten src/ --json
```

- 무엇을: OWASP Top 10 취약점 패턴 (XSS, CSRF, 인증 미흡 등)
- **Bandit**과 차이: Bandit은 Python 전용, Semgrep은 언어 무관 + 커스텀 룰 가능
- **Dev 2** 의미: Bandit이 못 잡는 패턴까지 이중 검사

---

pip-audit — CVE 취약점 검사

```
pip-audit --format=json
```

- 무엇을: 설치된 패키지의 알려진 보안 취약점(CVE) 검사
- **CVSS** 점수 기준: 7.0+ ⟶ Critical, 4.0~6.9 ⟶ Major
- **Dev 2** 의미: "이 라이브러리 버전에 해킹 가능한 취약점이 있다"


```
{
  "name": "requests", "version": "2.25.0",
  "vulns": [{"id": "CVE-2023-32681", "fix_versions": ["2.31.0"]}]
}
```


---

3. 도구별 역할 요약

도구	검사 대상	Dev 2 severity 기준
Pylint	코드 스타일/품질	대부분 Minor
radon	함수 복잡도	>10 ⟶ Major, >20 ⟶ Critical
Bandit	Python 보안 패턴	HIGH⟶Critical, MEDIUM⟶Major
Semgrep	OWASP 패턴	룰셋 기준
pip-audit	패키지 CVE	CVSS 7.0+⟶Critical

복습 퀴즈

1. Q: 정적 분석이 동적 테스트보다 빠른 이유는? A: (코드를 실행하지 않고 읽기만 해서 검사하기 때문)
2. Q: Bandit과 Semgrep의 차이는? A: (Bandit은 Python 전용. Semgrep은 언어 무관 + OWASP 룰셋 기반 패턴 검사)
3. Q: pip-audit이 필요한 이유는? A: (코드 자체가 아닌 사용 중인 라이브러리의 CVE 취약점을 검사하기 위해)